



# SISTEMA DE PIZZARIA

Documentação Técnica do Sistema

|             |                                         |
|-------------|-----------------------------------------|
| Disciplina  | Banco de Dados                          |
| Aluna       | Daiana Cristina Boudoux Pimentel Gomes  |
| Tecnologias | PostgreSQL 15+ • Java 17 • JDBC • Maven |
| Banco       | pizzaria                                |
| Versão      | 1.0 — Salvador, BA                      |

# PARTE 1 — SQL (Banco de Dados)

Esta seção descreve toda a estrutura e lógica implementada no banco de dados PostgreSQL, incluindo as tabelas, relacionamentos, views, functions e procedures.

## 1.1 Estrutura das Tabelas (DDL)

O banco é composto por 5 entidades que modelam o domínio de uma pizzaria:

| Tabela      | Descrição                                                                         |
|-------------|-----------------------------------------------------------------------------------|
| endereco    | Armazena endereços de forma independente, podendo ser compartilhados por clientes |
| cliente     | Cadastro de clientes com CPF único e vínculo opcional a um endereço               |
| pizza       | Cardápio de pizzas com nome e preço                                               |
| pedido      | Pedidos realizados, vinculados a um cliente, com data de criação                  |
| item_pedido | Tabela associativa que resolve o relacionamento N:N entre pedido e pizza          |

### Tabela: endereco

Entidade independente que centraliza endereços. Um endereço pode ser compartilhado por múltiplos clientes (cardinalidade 0,n do lado cliente).

```
CREATE TABLE endereco (  
  id_endereco SERIAL NOT NULL,  
  rua VARCHAR(100) NOT NULL,  
  numero VARCHAR(10) NOT NULL,  
  bairro VARCHAR(60) NOT NULL,  
  cidade VARCHAR(60) NOT NULL,  
  CONSTRAINT pk_endereco PRIMARY KEY (id_endereco)  
);
```

### Tabela: cliente

Cadastro de clientes. O campo cpf\_cliente possui restrição UNIQUE para evitar duplicatas. O id\_endereco é opcional (NULL permitido), refletindo a cardinalidade (0,1) — um cliente pode ou não ter endereço.

```
CREATE TABLE cliente (  
  id_cliente SERIAL NOT NULL,  
  nm_cliente VARCHAR(100) NOT NULL,  
  cpf_cliente CHAR(11) NOT NULL,  
  telefone_cliente VARCHAR(15) NULL,  
  id_endereco INT NULL,  
  CONSTRAINT pk_cliente PRIMARY KEY (id_cliente),  
  CONSTRAINT uq_cpf UNIQUE (cpf_cliente),
```

```

        CONSTRAINT fk_cli_end FOREIGN KEY (id_endereco)
            REFERENCES endereco (id_endereco)
            ON UPDATE CASCADE ON DELETE SET NULL
    );

```

### Tabela: pizza

Representa o cardápio. Cada pizza possui nome e preço. O tipo NUMERIC(8,2) garante precisão monetária.

```

CREATE TABLE pizza (
    id_pizza      SERIAL          NOT NULL,
    nm_pizza      VARCHAR(80)     NOT NULL,
    preco_pizza   NUMERIC(8, 2)   NOT NULL,
    CONSTRAINT pk_pizza PRIMARY KEY (id_pizza)
);

```

### Tabela: pedido

Registra os pedidos. Todo pedido obrigatoriamente pertence a um cliente (NOT NULL). A FK usa RESTRICT para impedir exclusão de clientes com pedidos vinculados.

```

CREATE TABLE pedido (
    id_pedido     SERIAL NOT NULL,
    data_pedido   DATE   NOT NULL,
    id_cliente    INT     NOT NULL,
    CONSTRAINT pk_pedido PRIMARY KEY (id_pedido),
    CONSTRAINT fk_ped_cli FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

```

### Tabela: item\_pedido

Tabela associativa que resolve o relacionamento N:N entre pedido e pizza. Armazena quantidade e preço unitário no momento do pedido — independente de alterações futuras no cardápio. A exclusão de um pedido cascadeia e remove automaticamente todos os seus itens.

```

CREATE TABLE item_pedido (
    id_itempedido SERIAL          NOT NULL,
    id_pedido      INT            NOT NULL,
    id_pizza       INT            NOT NULL,
    quantidade     INT            NOT NULL,
    preco_unitario NUMERIC(8, 2) NOT NULL,
    CONSTRAINT pk_item_pedido PRIMARY KEY (id_itempedido),
    CONSTRAINT fk_item_ped FOREIGN KEY (id_pedido)
        REFERENCES pedido (id_pedido)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_item_pizza FOREIGN KEY (id_pizza)
        REFERENCES pizza (id_pizza)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

```

## 1.2 Views

As views encapsulam consultas complexas com JOIN e agregação, expondo dados prontos para consumo pelo sistema Java e para relatórios.

### **vw\_pedidos\_completos**

Consolida pedidos com dados do cliente, quantidade de itens e valor total calculado. Usa GROUP BY com COUNT e SUM.

```
CREATE VIEW vw_pedidos_completos AS
SELECT p.id_pedido, p.data_pedido, c.nm_cliente, c.cpf_cliente,
       COUNT(ip.id_itempedido) AS qtd_itens,
       SUM(ip.quantidade * ip.preco_unitario) AS valor_total
FROM pedido p
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN item_pedido ip ON ip.id_pedido = p.id_pedido
GROUP BY p.id_pedido, p.data_pedido, c.id_cliente, c.nm_cliente,
         c.cpf_cliente, c.telefone_cliente;
```

### **vw\_itens\_pedido**

Detalha cada item com nome da pizza, cliente, quantidade, preço unitário e subtotal calculado. Usa 3 JOINS para cruzar item\_pedido, pedido, cliente e pizza.

### **vw\_pizzas\_mais\_vendidas**

Ranking de pizzas por volume de vendas e receita total gerada. Usa LEFT JOIN para incluir pizzas sem vendas, com ordenação DESC.

## 1.3 Functions

As functions realizam cálculos e consultas que retornam um valor, podendo ser chamadas diretamente em comandos SELECT.

| Function                         | Retorno                                                                     |
|----------------------------------|-----------------------------------------------------------------------------|
| fn_valor_total_pedido(id)        | NUMERIC — soma de (quantidade × preço_unitário) de todos os itens do pedido |
| fn_total_pedidos_cliente(id)     | INT — quantidade de pedidos realizados por um cliente                       |
| fn_pizza_mais_pedida_cliente(id) | VARCHAR — nome da pizza mais pedida por um cliente                          |

### Exemplo: fn\_valor\_total\_pedido

Calcula o valor total de um pedido somando todos os itens. Usa COALESCE para retornar 0 caso o pedido não tenha itens.

```
CREATE OR REPLACE FUNCTION fn_valor_total_pedido(p_id_pedido INT)
RETURNS NUMERIC(10, 2) LANGUAGE plpgsql AS $$
DECLARE v_total NUMERIC(10, 2);
BEGIN
    SELECT SUM(quantidade * preco_unitario) INTO v_total
    FROM item_pedido WHERE id_pedido = p_id_pedido;
    RETURN COALESCE(v_total, 0);
END;
$$;
```

Uso no Java:

```
SELECT fn_valor_total_pedido(?) -- parâmetro: id do pedido
```

## 1.4 Procedures

As procedures executam ações no banco de dados, podendo conter lógica de validação, inserção e exclusão. São chamadas via CALL no Java usando CallableStatement.

| Procedure              | Ação                                                              |
|------------------------|-------------------------------------------------------------------|
| sp_inserir_pedido      | Cria um novo pedido e retorna o ID gerado via parâmetro OUT       |
| sp_inserir_item_pedido | Adiciona item ao pedido com validações de existência e quantidade |
| sp_cancelar_pedido     | Remove pedido e todos os seus itens, com validação de existência  |

### sp\_inserir\_item\_pedido — validações implementadas

Esta procedure aplica três regras de negócio antes de inserir:

- Verifica se o pedido existe — RAISE EXCEPTION caso contrário
- Verifica se a pizza existe — RAISE EXCEPTION caso contrário
- Valida que a quantidade é maior que zero — RAISE EXCEPTION caso contrário

## 1.5 Consultas com JOIN (DML)

O sistema utiliza múltiplas consultas com JOIN para cruzar dados entre as tabelas:

| Consulta                            | JOINS utilizados                          |
|-------------------------------------|-------------------------------------------|
| Pedidos com dados do cliente        | pedido → cliente                          |
| Detalhamento completo de itens      | item_pedido → pedido → cliente → pizza    |
| Clientes com endereço               | cliente → endereco                        |
| Ranking de clientes por gasto total | cliente → pedido → item_pedido + GROUP BY |
| Pizzas com mais de 2 vendas         | pizza → item_pedido + HAVING              |

## PARTE 2 — Java (Sistema via Console)

O sistema Java se conecta ao banco PostgreSQL via JDBC e disponibiliza um menu interativo no console com operações de CRUD, realização de pedidos e geração de relatórios.

### 2.1 Estrutura do Projeto

O projeto segue o padrão de camadas Repository, separando a lógica de acesso ao banco da interface com o usuário:

| Pacote / Arquivo        | Responsabilidade                                                   |
|-------------------------|--------------------------------------------------------------------|
| ConexaoBD.java          | Gerencia a conexão JDBC com o PostgreSQL                           |
| model/                  | Classes que representam as entidades do banco (POJO)               |
| repository/             | Classes que executam SQL via PreparedStatement e CallableStatement |
| menu/MenuPrincipal.java | Interface de console com menus, cores ANSI e feedback ao usuário   |
| Main.java               | Ponto de entrada — instancia e inicia o MenuPrincipal              |

### 2.2 Conexão com o Banco (JDBC)

A classe ConexaoBD centraliza a criação da conexão. O driver PostgreSQL é carregado automaticamente pelo Maven (pom.xml).

```
public class ConexaoBD {
    private static final String URL =
"jdbc:postgresql://localhost:5432/pizzaria";
    private static final String USUARIO = "postgres";
    private static final String SENHA = "sua_senha";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, SENHA);
    }
}
```

Cada método dos repositories abre e fecha sua própria conexão usando try-with-resources, garantindo que nenhuma conexão fique aberta em caso de erro.

### 2.3 Camada Model

As classes model são POJOs (Plain Old Java Objects) que representam as entidades do banco. Cada atributo corresponde a uma coluna da tabela, com getters e setters. O método toString() formata os dados para exibição no console.

| Classe | Atributos principais |
|--------|----------------------|
|--------|----------------------|

|            |                                                               |
|------------|---------------------------------------------------------------|
| Endereco   | idEndereco, rua, numero, bairro, cidade                       |
| Cliente    | idCliente, nmCliente, cpfCliente, telefoneCliente, idEndereco |
| Pizza      | idPizza, nmPizza, precoPizza (BigDecimal)                     |
| Pedido     | idPedido, dataPedido (LocalDate), idCliente                   |
| ItemPedido | idItemPedido, idPedido, idPizza, quantidade, precoUnitario    |



## 2.4 Camada Repository (CRUD)

Cada repository é responsável por uma tabela e executa operações SQL via PreparedStatement, que previne SQL Injection ao usar parâmetros ? no lugar de concatenar strings.

### PizzaRepository — operações implementadas

| Método           | SQL executado                                                       |
|------------------|---------------------------------------------------------------------|
| listarTodas()    | SELECT id_pizza, nm_pizza, preco_pizza FROM pizza ORDER BY id_pizza |
| buscarPorId(id)  | SELECT ... FROM pizza WHERE id_pizza = ?                            |
| inserir(pizza)   | INSERT INTO pizza (nm_pizza, preco_pizza) VALUES (?, ?)             |
| atualizar(pizza) | UPDATE pizza SET nm_pizza = ?, preco_pizza = ? WHERE id_pizza = ?   |
| deletar(id)      | DELETE FROM pizza WHERE id_pizza = ?                                |

### PedidoRepository — uso de CallableStatement

O PedidoRepository usa CallableStatement para chamar as procedures do banco, incluindo o tratamento do parâmetro OUT que retorna o ID gerado:

```
// Chama a procedure sp_inserir_pedido e captura o ID gerado
CallableStatement cs = con.prepareCall("CALL sp_inserir_pedido(?, ?, ?)");
cs.setInt(1, pedido.getIdCliente());
cs.setDate(2, Date.valueOf(pedido.getDataPedido()));
cs.registerOutParameter(3, Types.INTEGER); // parâmetro OUT
cs.execute();
int idGerado = cs.getInt(3); // recupera o ID gerado pelo banco
```

### ItemPedidoRepository — validações via procedure

A inserção de itens é delegada à procedure sp\_inserir\_item\_pedido, que valida a existência do pedido e da pizza e garante quantidade positiva antes de inserir. Se alguma regra falhar, a procedure lança uma exceção que é capturada no Java e exibida ao usuário.

## 2.5 Uso das Views no Java

A classe RelatorioRepository consulta as três views diretamente via PreparedStatement, formatando os resultados em tabelas alinhadas no console:

| Método Java                | View / Function consultada                        |
|----------------------------|---------------------------------------------------|
| exibirPedidosCompleto()    | SELECT * FROM vw_pedidos_completos                |
| exibirItensDoPedido(id)    | SELECT * FROM vw_itens_pedido WHERE id_pedido = ? |
| exibirPizzasMaisVendidas() | SELECT * FROM vw_pizzas_mais_vendidas LIMIT 10    |

|                                |                                        |
|--------------------------------|----------------------------------------|
| exibirTotalPedido(id)          | SELECT fn_valor_total_pedido(?)        |
| exibirPizzaFavoritaCliente(id) | SELECT fn_pizza_mais_pedida_cliente(?) |

## 2.6 Interface de Console (UX/UI)

O MenuPrincipal aplica técnicas de UX para console usando códigos de escape ANSI, que colore e formatam o texto no terminal:

| Elemento                                                                                         | Aplicação                                            |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Bordas Unicode  | Delimita cabeçalhos de seção para hierarquia visual  |
| Verde ✓                                                                                          | Confirmação de operação bem-sucedida com ID gerado   |
| Vermelho ✗                                                                                       | Exibição de erros com mensagem descritiva            |
| Amarelo ⚠                                                                                        | Avisos e confirmações antes de ações destrutivas     |
| Ciano →                                                                                          | Indicador de campo de entrada do usuário             |
| Splash screen ASCII                                                                              | Tela inicial com logo da pizzaria                    |
| Pausar (Enter)                                                                                   | Evita que mensagens desapareçam antes de serem lidas |
| Listas automáticas                                                                               | Exibe clientes/pizzas/endereços antes de pedir o ID  |

## 2.7 Fluxo de um Pedido Completo

A sequência abaixo descreve o fluxo completo desde a criação do pedido até a consulta do total:

- Usuário acessa Pedidos → Novo pedido
- Sistema lista clientes disponíveis e solicita o ID
- Java chama `sp_inserir_pedido` via `CallableStatement` e recebe o ID do pedido criado
- Usuário acessa Adicionar item ao pedido e informa o ID do pedido
- Sistema exibe o cardápio completo e solicita ID da pizza e quantidade
- Java chama `sp_inserir_item_pedido` com validações automáticas no banco
- Usuário acessa Ver itens do pedido — o Java consulta `vw_itens_pedido` e `fn_valor_total_pedido`
- O valor total é exibido em tempo real, calculado diretamente no banco

## 2.8 Regras de Negócio Implementadas

| Regra                                      | Onde é aplicada                                            |
|--------------------------------------------|------------------------------------------------------------|
| CPF único por cliente                      | CONSTRAINT <code>uq_cpf</code> no banco (DDL)              |
| Cliente não pode ser removido com pedidos  | ON DELETE RESTRICT na FK <code>fk_ped_cli</code>           |
| Pizza não pode ser removida com itens      | ON DELETE RESTRICT na FK <code>fk_item_pizza</code>        |
| Quantidade do item deve ser maior que zero | Validação na procedure <code>sp_inserir_item_pedido</code> |

|                                       |                                                       |
|---------------------------------------|-------------------------------------------------------|
| Cancelar pedido remove todos os itens | ON DELETE CASCADE + procedure sp_cancelar_pedido      |
| Preço registrado no momento da compra | preco_unitario salvo em item_pedido, não referenciado |
| Confirmação antes de deletar          | Prompt s/n no MenuPrincipal.java                      |

— *Fim da Documentação* —